

Newton法

2次微分で一気にジャンプ

rkskmt

1

この資料について

- 本編「最適化と勾配降下法」の**参考資料**
- 勾配降下法の弱点（学習率の手動調整・収束の遅さ）を、**2次微分**で解決する手法
- ディープラーニングでは使われないが、**scikit-learn** のロジスティック回帰などの中で現役

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import japanize_matplotlib
4
5 plt.rcParams['font.size'] = 11
6 plt.rcParams['figure.dpi'] = 100
7
8 def f(x):
9     """目的関数 f(x) = x^2 - 4x + 3"""
10    return x**2 - 4*x + 3
11
12 def f_prime(x):
13     """1次微分 f'(x) = 2x - 4"""
14    return 2*x - 4
15
16 def f_double_prime(x):
17     """2次微分 f''(x) = 2"""
18    return 2
```

Python

2

なぜNewton法が必要か

勾配降下法の問題：

- 学習率を手動で調整
- 収束が遅い

解決策：もっと賢く進めないか？

① Newton法のアイデア

「傾き」だけでなく「曲がり具合」も使えば、**どこまで進めばいいか計算できる**のでは？

3

2次微分の役割

2次微分とは

1次微分: 傾き (どっちに進むか)

$$f'(x) = 2x - 4$$

2次微分: 曲率 (どれくらい曲がっているか)

$$f''(x) = \frac{d}{dx}(2x - 4) = 2$$

💡 2次微分の計算

1次微分をもう一度微分するだけ:

$$f'(x) = 2x - 4$$

$$f''(x) = 2 \times 1 - 0 = 2$$

ヘッセ行列 (Hessian Matrix) とは

注意: 1変数の場合、2次微分は単なる数値なので「ヘッセ行列」とは呼びません。

ヘッセ行列は2変数以上で使う用語です:

$$H = \begin{bmatrix} \frac{\partial^2 f}{\partial x^2} & \frac{\partial^2 f}{\partial x \partial y} \\ \frac{\partial^2 f}{\partial y \partial x} & \frac{\partial^2 f}{\partial y^2} \end{bmatrix}$$

① ノート

- 1変数: $f''(x)$ = 2次微分 (スカラー)
- 2変数以上: H = ヘッセ行列 (行列)

この資料は1変数なので「2次微分」と呼びます。

Newton法のアルゴリズム

更新式

$$x_{\text{new}} = x - \frac{f'(x)}{f''(x)}$$

- $f'(x)$: 1次微分（勾配）
- $f''(x)$: 2次微分（曲率）
- 学習率不要！

なぜこの式？

現在地点で関数を2次関数で近似：

$$f(x) \approx f(x_0) + f'(x_0)(x - x_0) + \frac{1}{2}f''(x_0)(x - x_0)^2$$

この2次関数の最小値を求めると：

$$x_{\text{min}} = x_0 - \frac{f'(x_0)}{f''(x_0)}$$

❗ 重要

2次関数の最小値は解析的に求められる → 一気にそこまでジャンプできる！

Newton法：手計算で実行

設定

- 関数: $f(x) = x^2 - 4x + 3$
- 1次微分: $f'(x) = 2x - 4$
- 2次微分: $f''(x) = 2$ (重要!)**
- 初期値: $x_0 = 0$

反復1

ステップ1：現在の位置

$$x = 0$$

ステップ2：関数値

$$f(0) = 0^2 - 4 \times 0 + 3 = 3$$

ステップ3：1次微分（勾配）

$$f'(0) = 2 \times 0 - 4 = -4$$

ステップ4：2次微分（曲率）

$$f''(0) = 2$$

ステップ5：NEWTON法で更新

更新式:

$$x_{\text{new}} = x - \frac{f'(x)}{f''(x)}$$

代入:

$$x_{\text{new}} = 0 - \frac{-4}{2}$$

計算:

$$= 0 - (-2)$$

$$= 0 + 2$$

$$= 2$$

❗ 重要

反復1の結果: $x = 0 \rightarrow x = 2$

反復2：収束確認

ステップ1：現在の位置

$$x = 2$$

ステップ2：関数値

$f(2) = 2^2 - 4 \times 2 + 3 = 4 - 8 + 3 = -1$

ステップ3：1次微分

$f'(2) = 2 \times 2 - 4 = 4 - 4 = 0$

① 収束！

勾配が0 → 極値に到達 → これ以上更新不要

答え

$x = 2, \quad f(2) = -1$
たった1回の反復で最適解に到達！

Newton法：Python実装

▶ コードを表示

```
Newton法で最小値を求める
=====

--- 反復 1 ---
現在の x = 0.0000
f(x) = 3.0000
f'(x) = -4.0000
f''(x) = 2.0000

更新：
    ステップ幅 = f'(x)/f''(x) = -4.0000/2.0000 = -2.0000
    x_new = 0.0000 - -2.0000 = 2.0000

--- 反復 2 ---
現在の x = 2.0000
f(x) = -1.0000
f'(x) = 0.0000
f''(x) = 2.0000

更新：
    ステップ幅 = f'(x)/f''(x) = 0.0000/2.0000 = 0.0000
    x_new = 2.0000 - 0.0000 = 2.0000

★ 収束！ f'(x) ≈ 0

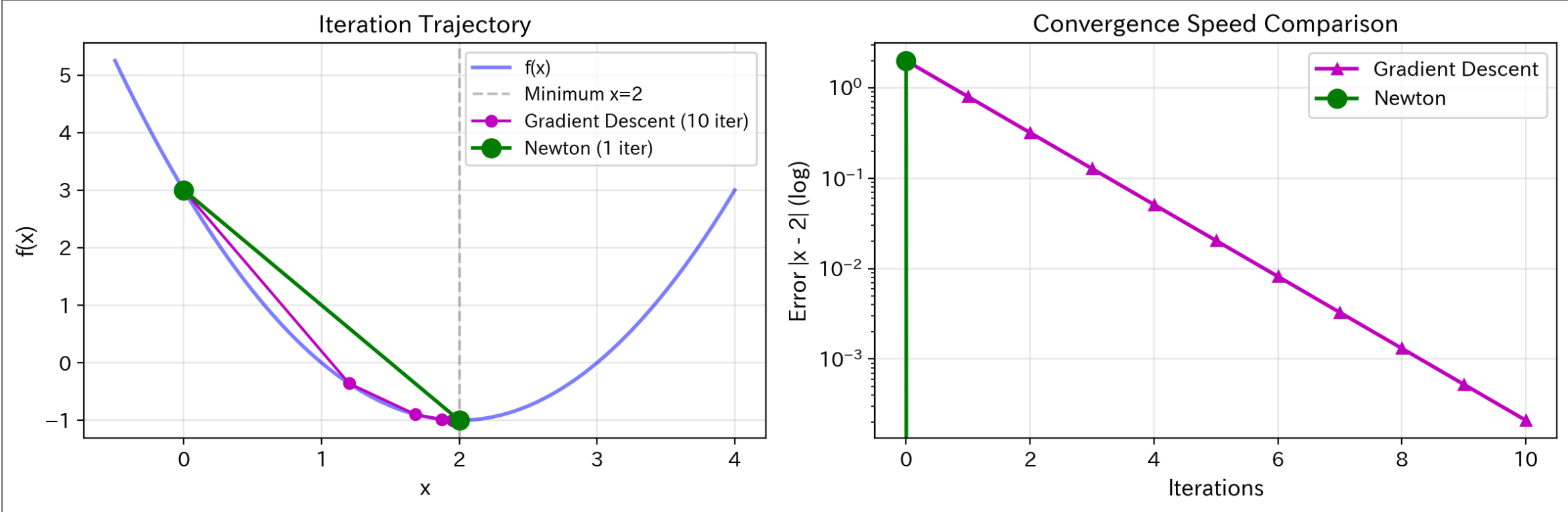
=====
最終結果： x = 2.0000, f(x) = -1.0000
```

💡 手で微分して関数定義

- $f(x) = x^2 - 4x + 3$
- $f'(x) = 2x - 4$
- $f''(x) = 2$

手で計算した結果を関数として定義します。自動微分は使いません。

可視化で比較



比較まとめ

数値比較

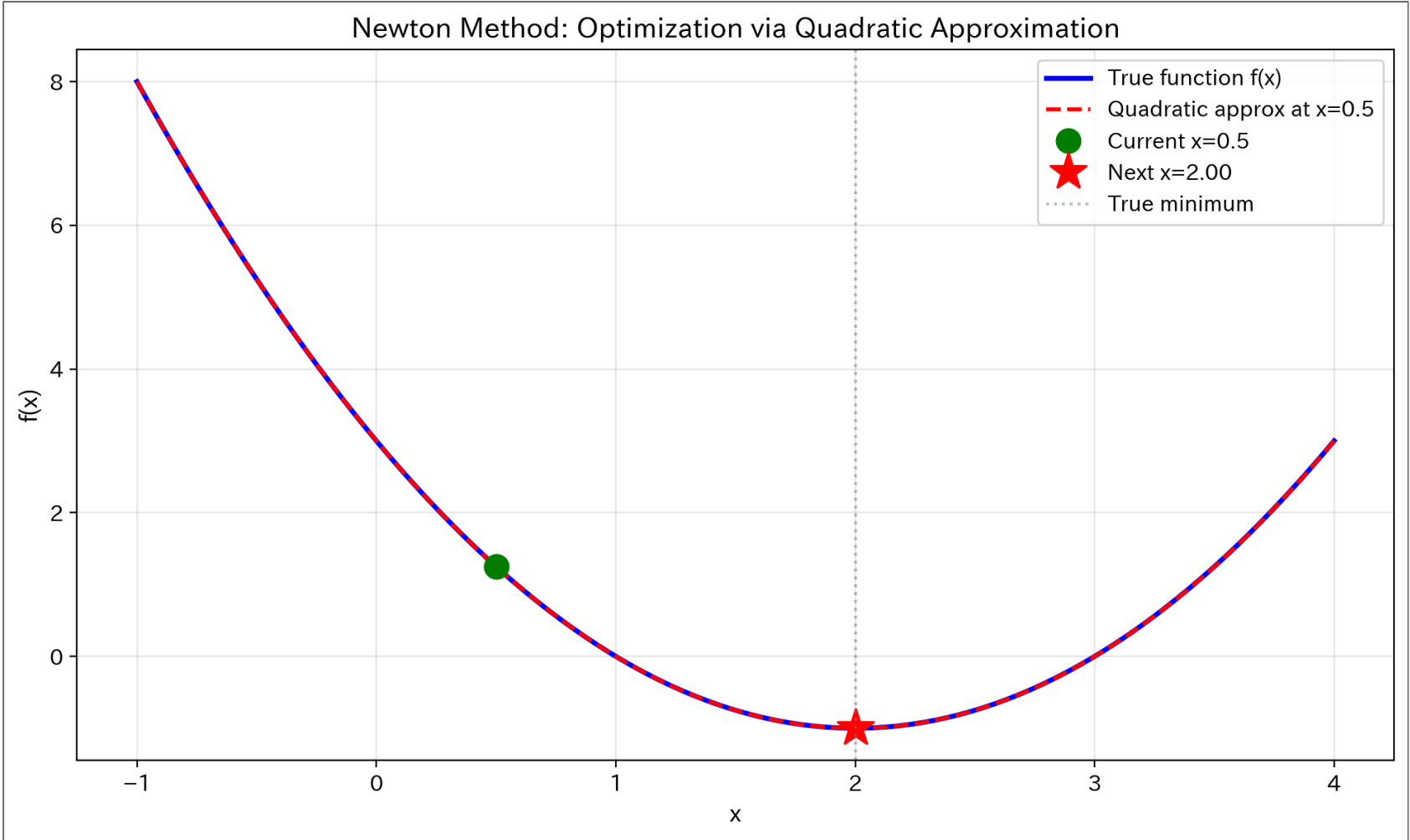
手法	反復回数	最終結果	誤差
勾配降下法 ($\alpha=0.3$)	10回	$x=1.9926$	0.0074
Newton法	1回	$x=2.0000$	0.0000

特徴比較

項目	勾配降下法	Newton法
使う情報	1次微分のみ	1次微分 + 2次微分
学習率	必要 (α)	不要
収束速度	遅い	非常に速い
1回の計算量	小	大
パラメータ調整	必要	不要
適用場面	大規模問題	小～中規模問題
実例	PyTorch、TensorFlow	scikit-learn

なぜNewton法は速いのか

視覚的説明



① 2次近似の威力

勾配降下法: 1次近似（接線）だけ → 「どっちに進むか」はわかるが「どこまで進むか」は手探り

Newton法: 2次近似（放物線） → 放物線の最小値を計算 → 一気にジャンプ

⚠ ただし、放物線がウソをつくとき...

この「当てはめた放物線へ一気にジャンプ」は、**当てはめた放物線が下に凸（お椀型）のとき** だけうまくいく。谷が複数ある関数では、上に凸の場所で放物線が逆さまになり、**山に登ったり反対側へ吹っ飛んだり** する。 → その失敗と直し方は **ローカルミニマムの罠** で詳しく扱う。

使い分けの指針

① どちらを使うべきか

勾配降下法を使う場合: - パラメータ数が多い（数万～数百万） - メモリが限られている - オンライン学習（データが逐次的に来る） - 例：ディープラーニング

Newton法を使う場合: - パラメータ数が少ない（数個～数千） - 高精度な解が必要 - バッチ学習 - 例：ロジスティック回帰、一般化線形モデル

練習問題

本編の練習問題と同じ関数 $f(x) = x^2 + 6x + 5$ について：

1. 2次微分を求めよ
2. 初期値 $x_0 = 0$ でNewton法を実行せよ
3. 反復1、反復2を手計算で示せ
4. 本編の勾配降下法 ($\alpha = 0.2$) の結果と比較して、何回で収束したか、どちらが速かったかを述べよ

13

解答例

2次微分:

$$f'(x) = 2x + 6, \quad f''(x) = 2$$

反復1:

$$x_{\text{new}} = 0 - \frac{f'(0)}{f''(0)} = 0 - \frac{6}{2} = -3$$

反復2 (収束確認) :

$$f'(-3) = 2 \times (-3) + 6 = 0$$

勾配が 0 なので収束。理論値 $x = -3$ に **1回** で到達した。勾配降下法は3回の反復後も $x = -2.352$ で、まだ収束していなかった。

14